

ALA-2

1. What is string immutability in Python?

Answer:

String immutability means once a string is created, it cannot be changed.

Syntax:

Python

```
s = "hello"
```

Example:

Python

```
s = "hello"
```

```
s = s + " world"
```

```
print(s) # hello world (new string created)
```

2. Why are Python strings immutable?

Answer:

To improve memory efficiency, security, and performance.

Syntax:

Python

```
a = "python"
```

```
b = "python"
```

Example:

Python

```
a = "python"
```

```
b = "python"
```

```
print(a is b) # True (same memory due to interning)
```

3. What happens when you try to modify a string?

Answer:

A new string object is created.

Syntax:

Python

```
s = s + "text"
```

Example:

Python

```
s = "hello"
```

```
s = s + " world"
```

```
print(s) # new string created
```

4. Can we change a single character of a string?

Answer:

No, it gives an error.

Syntax:

Python

```
s[index] = value
```

Example:

Python

```
s = "hello"
```

```
s[0] = "H" # TypeError
```

5. How can you modify a string indirectly?

Answer:

Using slicing or concatenation.

Syntax:

Python

```
new_string = value + old_string[index:]
```

Example:

Python

```
s = "hello"
```

```
s = "H" + s[1:]
```

```
print(s) # Hello
```

6. What type of error occurs when modifying a string directly?

Answer:

TypeError

Syntax:

Python

```
s[index] = value
```

Example:

Python

```
s = "hello"
```

```
s[0] = "H"
```

```
# TypeError: 'str' object does not support item assignment
```

7. How does immutability help in memory optimization?

Answer:

Python reuses string objects (interning).

Syntax:

Python

```
a = "text"
```

```
b = "text"
```

Example:

Python

```
a = "hello"
```

```
b = "hello"
```

```
print(a is b) # True (same memory)
```

8. Are all data types in Python immutable like strings?

Answer:

No, some are mutable.

Syntax:

Python

```
list_name[index] = value
```

Example:

Python

```
lst = [1, 2, 3]
```

```
lst[0] = 10
```

```
print(lst) # [10, 2, 3]
```

9. What is the difference between mutable and immutable objects?

Answer:

Mutable can change, immutable cannot.

Syntax:

Python

Mutable

```
list[index] = value
```

Immutable

```
string[index] = value # error
```

Example:

Python

Mutable

```
lst = [1, 2]
```

```
lst[0] = 5
```

Immutable

```
s = "hi"
```

```
# s[0] = "H" # Error
```

10. Why are immutable strings useful in multi-threading?

Answer:

They are safe because they cannot be changed.

Syntax:

Python

```
# Same string accessed by multiple threads
```

Example:

Python

```
s = "hello"
```

```
# multiple threads can read safely
```

```
print(s)
```

If you want, I can also make this into a short revision chart or PDF notes for your exams ☐.